

Assignment 1 - Advanced Programming 2026

Anton Bang Lejbølle & Jonnas Dennis Hansen

September 2026

1 Introduction

We found the description of the for loop implementation a bit ambiguous especially in the case where p and i are the same. Specifically should binding variable p to v make the for loop incrementer start from v instead of from 0? The problem text also says "While $i < n$: evaluate expression body and bind p to the result. Increment i ". So if p and i are the same then result is bound to i , should this change the value of the incrementer? And what should be returned when the for loop ends because both the result of the body expression and the incremented i is bound to p ?

We decided to implement the for loop such that the value of the incrementer cannot be changed, there by ensuring that the for loop will always run n times. This has the effect that if i and p are the same, then if there are no errors, the for loop will always return the bound n .

It was not specified how the `(Var vname)` expression should be printed, so we decided for it to just return `vname`.

We consider the quality of our solution to be good as we have a fully functional codebase in which we apply rigorous testing to the functionality of our program to prove to ourselves that our solution is of a good standard.

Our solution is fully functional, and passes all our test cases. Our tests cover basic arithmetic operations, conditional operations, variables, `Let`, `ForLoop`, `Lambda`, `Apply`, and `TryCatch` as well as the pretty printing of each of these expression types. We are testing both successful evaluations and expected error cases. We therefore believe our solution is fully functional and correctly implemented.

2 Answers to questions

1. What is the observable evaluation order for `Apply`, what difference does it make, and which of your test(s) demonstrate this?

The observable evaluation order for `Apply` is function expression first and then argument expression and then the function body at the end. This makes a

difference because the argument is not evaluated if the function fails. Therefore if both returns errors, the function error is returned. This is demonstrated in this test

Listing 1: Test case for Apply when the function expression fails

```
testCase "Apply_first_expression_errors" $
  eval envEmpty
    (Apply
      (Div (CstInt 5) (CstInt 0))
      (Var "SomeString"))
    @?= Left "Division_by_zero",
```

2. Would it be a correct implementation of eval for TryCatch to first call eval on both subexpressions? Why or why not? If not, under which assumptions might it be a correct implementation?

In general no, TryCatch should evaluate the try branch first and only evaluate the catch part if the try branch fails. This matters because if the catch branch contains an infinite loop then the try branch will never get terminated.

Evaluating both expressions could be correct if the evaluation has no side effects and both expressions are guaranteed to terminate. Then evaluating the second expression does not change the result.

3. Is it possible for your implementation of eval to go into an infinite loop for some inputs?

Yes it is possible for our implementation of eval to go into an infinite loop for some inputs.

An example is the provided fact and Y combinator functions that are used to demonstrate recursion. If you change the 0 in the if statement `n==0` to a value that `n` never reaches, for example 1 above the starting value of `n`. Then the if statement never becomes true and it will result in an infinite loop.

4. For each of the following equalities, explain under which circumstances they hold.

a) `Apply (Lambda "v" x) y == x`

This equality holds when `v` is not used in `x` and `y` evaluates successfully.

b) `TryCatch (TryCatch x y) z == TryCatch x y`

This equality holds when `y` never fails such that `z` will never get called.

5. Suppose we wanted APL to be dynamically scoped. What would need to change in your implementation?

To make the language dynamically scoped, the lambda should no longer store the environment in which it was created. Instead, when the function is applied, its body should be evaluated using the environment at the point of the function call.

6. What is the asymptotic complexity of printExp on a left-nested expression such as Add (Add (Add ...) ...) ...?

Strings in haskell are implemented as linked lists of characters. This means that it is $O(1)$ to append a char to the front of the string, but doing string concatenation with ++ is $O(n)$ where n is the length of the first string.

Our implementation uses string concatenation, so in a left nested expression such as Add (Add (Add ...) ...) ... with m Adds, there will be in the order of m string concatenations where the m 'th string concatenation will take $O(m)$ as the first string will have $m-1$ adds.

This means that the asymptotic complexity of printExp on a left-nested expression such as Add (Add (Add ...) ...) ... with m Adds will take $O(m^2)$

7. State properties that it would be natural for printExp to satisfy (you are encouraged to hypothesize additional machinery in order to state such properties).

It would be natural for printExp to run in linear time ($O(n)$).

It would be natural for the result of printExp to be able to run and give the same result as eval.

8. Give a reason for why Val is defined in APL.Eval rather than in APL.AST.

You could implement another evaluation module that also uses the definition of APL from APL.AST, but handles the evaluation in a different way that does not use Val. So because Val is used in the evaluation and not in the definition of the language, it should be defined in APL.EVAL rather than in APL.AST.